

FichajeApp

Manual de código

Proyecto del CFGS DAM

Alumno: Jaime Rodríguez Rafael

Tutora: Judit Tardajos González

Curso: 2025/2026

Entrega: 12/2025

Índice.

1. Introducción……..... 1

2. Frontend ……..... 4

3. Comunicación Frontend – Backend – Base de datos 16

4. Backend ……..... 26

5. Base de datos ……..... 34

1. INTRODUCCIÓN

Este manual de código documenta la implementación completa del sistema FichajeApp, una aplicación web desarrollada para la gestión de fichajes de entrada/salida y control de ausencias de empleados.

1.1 Tecnologías Utilizadas

Frontend:

- **Angular 18 (Standalone Components).** Framework principal (standalone Componentes.
- **TypeScript 5.x.** Lenguaje de programación.
- **RxJS**. Programación reactiva (Observables).
- **SCSS**. Preprocesador CSS.
- **Bootstrap 5.** Framework CSS responsive.
- **Ng-Zorro (componentes UI).** Componentes UI (Ant Desingn).

Backend:

- **ASP.NET Core 8**. API REST.
- **C# 12.** Leguaje backend.
- **Entity Framework Core**. ORM (acceso a BD).
- **PostgreSQL 15+.** Base de datos relacional.
- **JWT.** Autenticación stateless.
- **BCrypt.** Hash de contraseñas

Herramientas de desarrollo:

- **VS Code. Editor principal.**
- **Angular CLI. Generación de código y build.**
- **Git. Control de versiones.**
- **Npm. Gestor de paquetes.**

1.2 Arquitectura general

DIAGRAMA DE CLASES

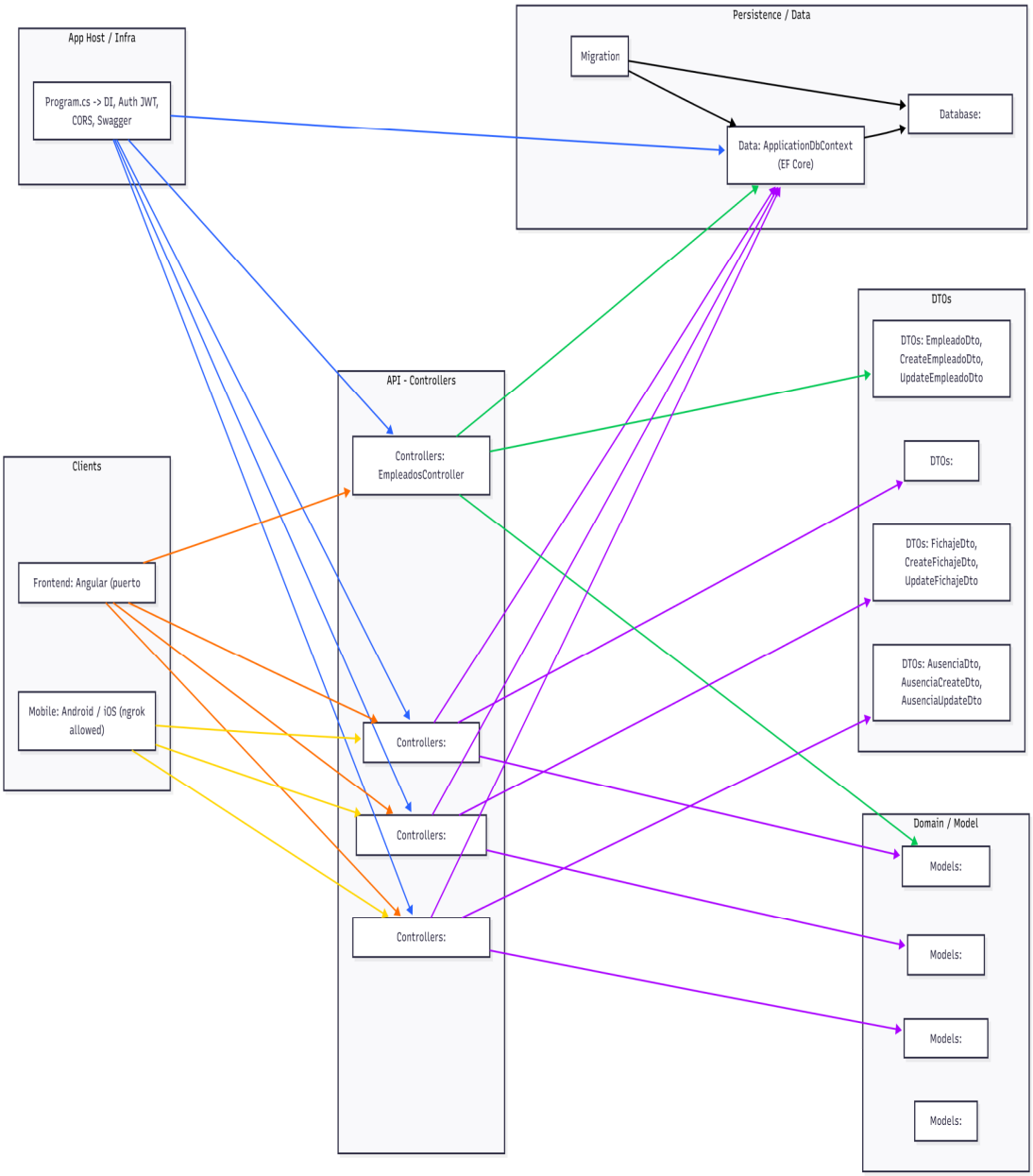
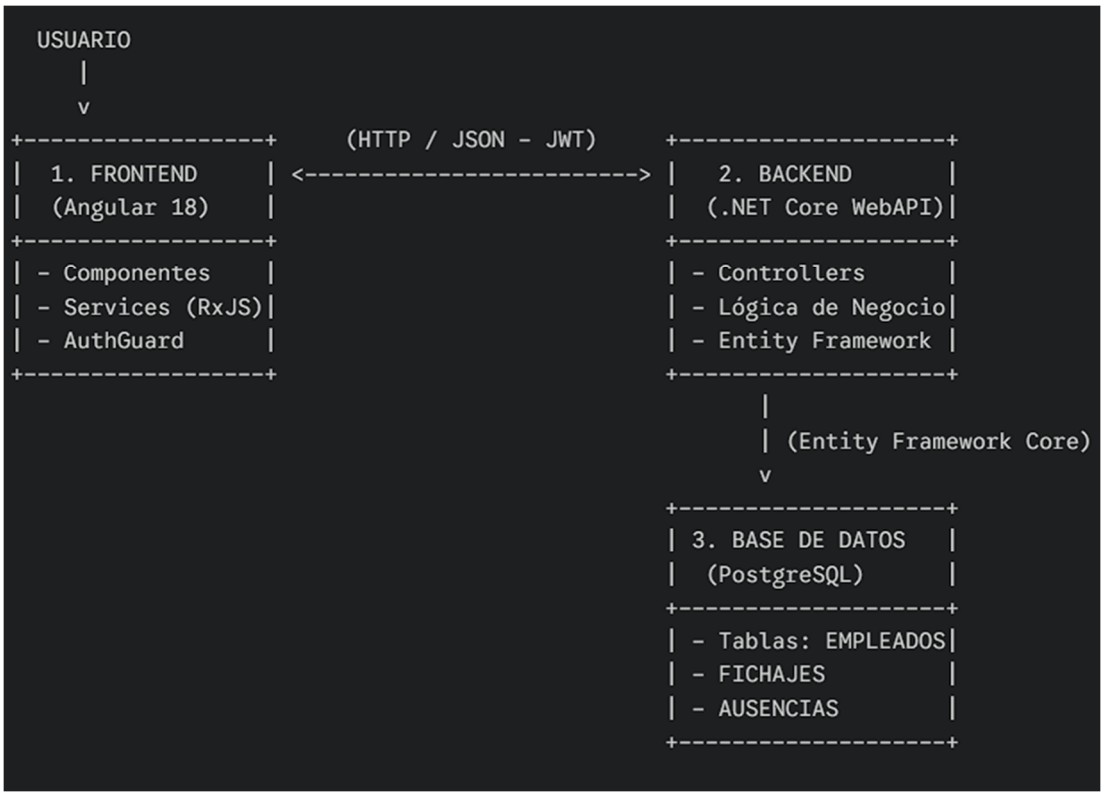


Diagrama ASCII de la arquitectura del proyecto



2. Frontend

El frontend ha sido desarrollado con **Angular**, utilizando *Standalone Components*, y contiene todos los elementos necesarios para mostrar la interfaz de la aplicación.

Cada componente de la interfaz está formado por al menos tres archivos esenciales:

- Un archivo **TypeScript (.ts)**, que contiene la lógica.
- Un archivo **HTML (.html)**, para la parte visual (plantilla).
- Un archivo de **estilos SCSS (.scss)**, para el formato visual.

La organización de los componentes en la capa visual se realiza principalmente a través de las carpetas components y pages.

Componentes Visuales

Carpeta components

- **Header:** Contiene el encabezado de la aplicación. Es dinámico, variando según la página y el **rol del usuario**. Incluye los accesos a "Fichaje", "Ausencias", "Gestión de empleados" y el botón de "Cerrar sesión".

```
@Component({
  selector: 'app-header',
  standalone: true,
  imports: [CommonModule, RouterModule, RouterLink],
  templateUrl: './header.html',
  styleUrls: ['./header.scss']
})
export class Header {
  isAdmin$: Observable<boolean>;
  isLogged$: Observable<boolean>;

  constructor(private auth: AuthService, private router: Router)
{
    this.isAdmin$ = this.auth.isAdmin$;
    this.isLogged$ = this.auth.isLogged$;
  }

  logout(): void {
    this.auth.logout();
```

```
    // Limpiar también el historial de fichajes local
    localStorage.removeItem('fichaje_history_v1');
    this.router.navigateByUrl('/login', { replaceUrl: true
}).then(() => {
      // Forzar recarga completa para limpiar estado en memoria
      window.location.reload();
    });
  }
}
```

Código HTML del componente header:

```
<header>
  <nav>
    <!-- Logo y enlace a la página principal -->
    <div class="logo">
      <a routerLink="/" class="logo-link">
        <img src="/Logo_FichajeApp.png" alt="Logo de FichajeApp"
/>
        <h2>FichajeApp</h2>
      </a>
    </div>

    <!-- Enlaces de navegación -->
    <div class="header-links">
      <!-- <a routerLink="/" routerLinkActive="active"
[routerLinkActiveOptions]="{ exact: true }">Inicio</a> -->

      <!-- Mostrar enlaces solo si hay sesión -->
      <ng-container *ngIf="isLogged$ | async">
        <a routerLink="/fichaje"
routerLinkActive="active">Fichaje</a>
        <a routerLink="/ausencias"
routerLinkActive="active">Ausencias</a>
        <a *ngIf="isAdmin$ | async" routerLink="/registro"
routerLinkActive="active">Gestión de empleados</a>
        <a (click)="logout()" style="cursor: pointer">Cerrar
sesión</a>
      </ng-container>
    </div>
  </nav>
</header>
```

- **Footer: Contiene el pie de página estático de la aplicación. Incluye los accesos a "Privacidad" y "Contacto".**

Código ts del componente footer:

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-footer',
  imports: [RouterModule],
  templateUrl: './footer.html',
  styleUrls: ['./footer.scss']
})
export class Footer {
}

Código HTML del componente footer:
<!-- Footer de la aplicación -->
<footer class="footer">
  <h3 class="copyright">© 2025 FichajeApp - Todos los derechos
reservados</h3>

  <div class="footer-links">
    <a routerLink="/privacidad"
routerLinkActive="active">Privacidad</a>
    <a routerLink="/contacto"
routerLinkActive="active">Contacto</a>
  </div>
</footer>
```

Carpeta pages

- **Ausencias:** Contiene la lógica y la vista para solicitar ausencias y mostrar las ausencias de los usuarios.
- **Contacto:** Página para contactar con la empresa.
- **Fichaje:** Componente principal para realizar fichajes, consultar el historial y registrar las ausencias.
- **Login:** Página de inicio de la aplicación y gestión de la autenticación de usuarios.
- **Privacidad:** Contiene la Política de privacidad de la aplicación.
- **Registro:** Componente con la lógica y la vista para la gestión de nuevos usuarios (registro, modificación y eliminación), **restringido solo para Administradores**.

Capa de Servicios (services)

La carpeta services contiene archivos de Angular que actúan como la **capa de abstracción** entre los componentes del frontend y la API del backend.

La función principal de estos servicios es:

- **Centralizar la comunicación HTTP con la API del backend (.NET).**
- **Gestionar el estado global de la aplicación (autenticación).**
- **Encapsular la lógica de negocio de acceso a datos.**
- **Proporcionar tipos de TypeScript (interfaces/DTOs) para type-safety (seguridad de tipos).**
- **Reutilizar código y facilitar la testabilidad y el mantenimiento.**

Esta carpeta contiene 4 servicios que son los siguientes:

- Auth.service.ts: Controla el estado de sesión del usuario. Guarda el token JWT, rol y empleadoId en localStorage. Proporciona observables reactivos (isLogged$, isAdmin$). Usa BehaviorSubject para que los componentes reaccionen automáticamente.

  Sus métodos principales son:

  - setAuth(token, rol, empleadoId), guarda la sesión tras el login.
  - Logout(), cierra la sesión y limpia localStorage.
  - IsLogged$, observable para saber si hay sesión activa.
  - IsAdmin$, observable para saber si es administrador.

- Ausencias.service.ts: Gestiona el CRUD de ausencias (vacaciones, permisos...).

  - Contiene los Endpoints:
    - crear(dto), POST /api/Ausencias para solicitar ausencias.
    - Listar(), GET /api/Ausencias/mias, para ver las ausencias del usuario logueado.
    - ListarPorEmpleado(id), GET /api/Ausencias/usuario/{id}, para consultar las ausencias de los usuarios si eres Admin.
    - Borrar(id), DELETE /api/Ausencias/{id}, para eliminar ausencias.

- ActualizarEstado(id, aceptada), PUT /api/Ausencias/{id}/aprobar, para aprobar/denegar ausencias si eres Admin.

- Dtos:
  - CreateEmpleadoDto – Crear empleado (con contraseña).
  - UpdateEmpleadoDto – Actualizar empleado (campos opcionales).
  - EmpleadoDto - Gestión de Fichajes.
  - Empleados.service.ts: Gestiona los CRUD de empleados, (solo Admin).
    - Contiene los Endpoints:
      - create(dto), POST /api/Empleados para registrar un nuevo empleado.
      - getAll(), GET /api/Empleados, para listar a todos los empleados.
      - update(id, dto), PUT /api/Empleados/{id}, para actualizar a los empleados.
      - delete(id), DELETE /api/Empleados/{id}, para eliminar empleados.
    - Dtos:
      - CreateEmpleadoDto – Crear empleado (con contraseña).
      - UpdateEmpleadoDto – Actualizar empleado (campos opcionales).
      - EmpleadoDto – Empleado completo del backend.
  - Fichajes.service.ts: Gestiona los fichajes, entradas/salidas y consultar historial.
    - Contiene los Endpoints:
      - crear(dto), POST /api/Fichajes para fichar entrada y salida.
      - misFichajes(limit?), GET /api/Fichajes/mios, para ver el historial de fichajes del usuario logueado.
      - todos(), GET /api/Fichajes?all=true, para consultar los fichajes de los usuarios si eres Admin.
      - actualizarObservacion(id, obs), PUT /api/Fichajes/{id}/observación, para añadir una incidencia al fichaje realizado.
    - Dtos:
      - FichajeCreateDto – Crear fichaje (Entrada/Salida).
      - FichajeDto – Fichaje completo del backend.

        - TipoFichaje – Tipo de fichaje “Entrada” | “Salida”.

Con esta arquitectura:

- Separamos las responsabilidades, los componentes NO hacen peticiones HTTP directamente.
- Type-safety, la interfaces TypeScript previenen errores.
- Reutilización, multiples components usan el mismo servicio.
- Testabilidad, ´fácil mockear servicios en tests.
- Mantenimiento, los cambios en API solo afectan a los servicios.
- Interceptores automáticos, JWT se añade sin código extra.

Archivos de configuración de Angular

Los archivos de configuración que ya vienen con Angular son:

- App.config.ts: Define los providers globales de la aplicación. Es el punto central donde se configuran todos los servicios y funcionalidades que Angular necesita para funcionar. Es como el corazón de la configuración, si falta algún provider, partes de la app dejarán de funcionar.
- App.html: Es la plantilla HTML del componente principal (App). Define la estructura base que se ve en todas las páginas. Contiene:
  - <app-header>: Componente del encabezado.
    - Barra de navegación.
    - Visible en todas las páginas.
    - Contiene logo, menú, botón de logout.
  - <main>: Punto de inserción dinámico.
    - Angular Router carga aquí el componente de cada ruta como /Login, /fichaje o /ausencia.
  - <app-footer>: Componente del pie de página.
    - Links a Contacto, Privacidad.
    - Copyright e información legal.
    - Siempre está al final de la página.

      Define el layout base de toda la aplicación. Header y Footer son constantes, solo cambia el contenido del <router-outlet>.

```
<app-header></app-header>
<main>
 <router-outlet></router-outlet>
</main>
<app-footer></app-footer>
```

- App.scss: Estilos del componente principal. Define los estilos específicos del componente App. Controla el layout flex para que el footer siempre esté al fondo.

- App.ts: Lógica del componente principal. Contiene la lógica TypeScript del componente raíz. Calcula dinámicamente las alturas del header y footer para ajustar el espaciado del main. Contiene:

```
@Component({
 selector: 'app-root',
 standalone: true,
 imports: [CommonModule, RouterOutlet, Header, Footer],
 templateUrl: './app.html',
 styleUrls: ['./app.scss']
})
```

    - Standalone: true, no necesita app.module
    - Imports, Los componentes y directivas que usa.

- Styles.scss: Estilos globales de la aplicación, en el se importa la biblioteca de ng-zorro.

```
{
 box-sizing: border-box;
 margin: 0;
 padding: 0;
}
```

Elimina márgenes/paddings por defecto del navegador,

Layout principal: Body como contenedor flex vertical, Main crece automáticamente y Padding-top para compensar header fijo.

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
main {
  flex: 1 0 auto; // Crece para empujar footer
  padding-top: var(--app-header-height);
  padding-bottom: 20px;
}
```

Responsive Design:

- **Tablets.**

```
@media (max-width: 992px) {
  .page-header-box h1 { font-size: 32px; }
}
```

- **Móviles:**

```
@media (max-width: 600px) {
  .page-header-box h1 { font-size: 24px; }
  .page-header-box { padding: 10px 16px; }
}
```

- **FullHD (1920px+):**

```
@media (min-width: 1920px) {
  .page-header-box { max-width: 900px; }
  main { padding-left: 5%; padding-right: 5%; }
  .card-body { font-size: 18px; }
}
```

- **2K/4K (2560px+):**

```
@media (min-width: 2560px) {
  .page-header-box { max-width: 1000px; }
   .card-body { font-size: 20px; }
  .btn-lg { font-size: 20px; padding: 14px 24px; }
}
```

Es el estilo visual de toda la aplicación. Define colores, espaciados, tipografía y comportamiento responsive.

Archivos de seguridad de rutas:

Los archivos de seguridad de rutas (Guards):

- auth.guard.ts: Protege rutas que requieren Login, para que no pueda acceder nadie con solo escribir la ruta en el navegador.

```
/**
 * Guard de Autenticación
 *
 * Protege rutas que requieren que el usuario esté logueado.
 * Si alguien intenta acceder sin haberse logueado, lo redirige
al login.
 *
 * Se usa en el archivo de rutas (app.routes.ts) con canMatch.
 * Por ejemplo, las rutas de fichaje, ausencias y registro están
protegidas.
 */

import { inject } from '@angular/core';
import { CanMatchFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
```

```
export const authGuard: CanMatchFn = () => {
  // Inyectar servicios necesarios
  const auth = inject(AuthService);
  const router = inject(Router);

  // Si hay token (está logueado), permitir acceso
  if (auth.hasToken()) return true;

  // Si no hay token, redirigir al login y denegar acceso
  router.navigateByUrl('/login', { replaceUrl: true });
  return false;
};
```

- admin.guard.ts:.Protege rutas solo para administradores, para que los usuarios que tienen claves, pero no permisos a esa parte de la aplicación, no puedan acceder.

```
/**
 * Guard de Administrador
 *
 * Protege rutas que solo pueden acceder usuarios con rol Admin.
 * Si un usuario normal (Empleado) intenta acceder, lo redirige
al inicio.
 *
 * Se usa en el archivo de rutas (app.routes.ts) con canMatch.
 * Por ejemplo, la ruta de registro de empleados solo la puede
ver un Admin.
 */

import { inject } from '@angular/core';
import { CanMatchFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const adminGuard: CanMatchFn = () => {
  // Inyectar servicios necesarios
  const auth = inject(AuthService);
  const router = inject(Router);

  // Comprobar si el usuario es Admin
  let isAdmin = false;
  auth.isAdmin$.subscribe(v => isAdmin = v).unsubscribe(); //
Suscripción instantánea y cancelación

  // Si es Admin, permitir acceso
  if (isAdmin) return true;
```

```
  // Si no es Admin, redirigir al inicio y denegar acceso
  router.navigateByUrl('/', { replaceUrl: true });
  return false;
};
```

Interceptor (AuthInterceptor):

Añade JWT automáticamente a todas las peticiones, gestiona errores 401 (token expirado) y es la plieza clave de la comunicación con el backend.

Rutas (app.routes.ts):

En este archivo está la configuración de routing, lazy loading de componentes y el uso de guards en rutas protegidas

```
/**
 * CONFIGURACIÓN DE RUTAS DE LA APLICACIÓN
 *
 * Aquí se definen todas las rutas (URLs) de la aplicación y qué
componente
 * se carga en cada una. También se configuran los guards que
protegen
 * ciertas rutas para que solo puedan acceder usuarios
autenticados o admins.
 *
 * Tipos de rutas:
 * - Públicas: Cualquiera puede acceder (login, contacto,
privacidad)
 * - Protegidas: Solo usuarios logueados (fichaje, ausencias)
 * - Admin: Solo usuarios con rol Admin (registro de empleados)
 */

import { Routes } from '@angular/router';
import { authGuard } from './guards/auth.guard';
import { adminGuard } from './guards/admin.guard';

import { Login } from './pages/login/login';
import { Registro } from './pages/registro/registro';
import { Privacidad } from './pages/privacidad/privacidad';
import { Ausencias } from './pages/ausencias/ausencias';
import { Contacto } from './pages/contacto/contacto';
import { Fichaje } from './pages/fichaje/fichaje';
```

```
export const routes: Routes = [
  // Ruta pública: página de inicio de sesión
  { path: 'login', component: Login },

  // ============================================
  // RUTAS PROTEGIDAS (requieren estar logueado)
  // ============================================

  // Página de fichaje (entrada/salida)
  { path: 'fichaje',   component: Fichaje,   canMatch:
[authGuard] },

  // Página de gestión de ausencias
  { path: 'ausencias', component: Ausencias, canMatch:
[authGuard] },

  // ============================================
  // RUTAS SOLO PARA ADMIN
  // ============================================

  // Página de registro/gestión de empleados (solo Admin)
  // Requiere estar logueado Y ser Admin
  { path: 'registro',  component: Registro,  canMatch:
[authGuard, adminGuard] },

  // ============================================
  // RUTAS PÚBLICAS (sin protección)
  // ============================================

  // Página de política de privacidad
  { path: 'privacidad', component: Privacidad },

  // Página de contacto
  { path: 'contacto',   component: Contacto },

  // Redirección legacy (por si alguien pone .html)
  { path: 'contacto.html', redirectTo: 'contacto', pathMatch:
'full' },

  // ============================================
  // RUTAS POR DEFECTO Y ERROR
  // ============================================

  // Ruta raíz: redirige al login
  { path: '', redirectTo: 'login', pathMatch: 'full' },

  // Ruta comodín: cualquier URL no definida vuelve al login
  { path: '**', redirectTo: '' }
];
```

3. COMUNICACIÓN FRONTEND ↔ BACKEND ↔ BASE DE DATOS.

Funcionamiento del proxy:

1. El frontend hace una petición a, /api/auth/login (URL relativa)
2. El navegador construye la URL completa: http://localhost:4200/api/auth/login
3. El proxy de Angular intercepta cualquier petición que empiece con, /api
4. Reescribe la URL a, https://localhost:7286/api/auth/login
5. El navegador recibe la respuesta como si viniera del mismo origen (localhost:4200)
6. No hay error CORS porque el navegador no sabe que es otro servidor

Ventajas del proxy:

- URLs relativas en el código (más limpio y mantenible)
- Sin configuración CORS en el backend durante desarrollo
- Mismo código funciona en producción (donde frontend y backend están en el mismo dominio)
- Facilita debug con logLevel: "debug" en la configuración

El proxy solo existe en desarrollo. En producción, el frontend compilado y el backend se sirven desde el mismo dominio (ej: https://fichajeapp.com para Angular y https://fichajeapp.com/api para ASP.NET).

AuthInterceptor: Inyección Automática de JWT

El **AuthInterceptor** es un componente fundamental de la seguridad del frontend. Su función es interceptar todas las peticiones HTTP salientes y añadir automáticamente el token JWT en el header Authorization.

Ubicación: auth.interceptor.ts

Funcionamiento:

1. **Lectura del token:** El interceptor lee el token JWT almacenado en localStorage con la clave jwt_token.

2. **Clonación de la petición:** Las peticiones HTTP en Angular son inmutables, por lo que el interceptor clona la petición original y le añade el header Authorization: Bearer <token>.
3. **Peticiones públicas:** Si no hay token (por ejemplo, en el login), la petición se envía sin modificar.
4. **Manejo de expiración:** Si el backend responde con código 401 (Unauthorized) y había un token, significa que el token expiró. El interceptor automáticamente:
    - Llama a AuthService.logout() para limpiar localStorage
    - Redirige al usuario a /login
    - Propaga el error para que el servicio pueda manejarlo

Ventajas del interceptor:

- No hay que añadir el JWT manualmente en cada servicio (código limpio)
- Lógica centralizada en un solo archivo (fácil mantenimiento)
- Logout automático cuando el token expira (mejor UX)

Flujo Completo: Ejemplo Fichaje de Entrada

A continuación, se describe el flujo completo de datos cuando un empleado registra su entrada al trabajo.

Fase 1: Interacción del Usuario (Frontend)

Paso 1

El usuario hace click en el botón "Fichar Entrada" en la interfaz. Este evento ejecuta el método fichar('ENTRADA') del componente FichajeComponent.

Paso 2

Validaciones previas: El componente verifica que el último fichaje fue una salida. Si la última acción fue una entrada sin salida, se muestra un error y no se permite continuar (validación de UX).

Paso 3

Registro local temporal: Antes de enviar al servidor, el componente crea un registro temporal en el array history[] y lo guarda en localStorage. Esto mejora la experiencia del usuario porque ve el resultado inmediatamente, aunque el servidor tarde en responder.

Paso 4

Llamada al servicio: El componente llama al método FichajesService.crear() pasando un DTO (Data Transfer Object) con los datos:

- tipo: "Entrada"
- fechaHora: timestamp actual en formato ISO (ej: "2025-12-05T09:30:00.000Z")
- observacion: texto opcional de incidencia (si el usuario lo escribió)

El DTO NO incluye el empleadoId. Esto es crítico para la seguridad, ya que si el frontend enviara el ID, un usuario malicioso podría modificarlo con DevTools y fichar por otro empleado. El backend obtendrá el empleadoId del token JWT, que no puede ser manipulado.

Fase 2: Petición HTTP (Frontend)

Paso 5

HttpClient envía POST: El servicio FichajesService usa HttpClient.post() para enviar la petición:

La URL /api/Fichajes es relativa, lo que permite que funcione tanto en desarrollo (con proxy) como en producción (mismo dominio).

Paso 6

AuthInterceptor intercepta: Antes de que la petición salga del navegador, el authInterceptor la intercepta:

- Lee el token de localStorage.getItem('jwt_token')
- Clona la petición HTTP (son inmutables)
- Añade el header Authorization: Bearer <token>
- Deja pasar la petición modificada

Paso 7

Proxy redirige: La petición llega al proxy de Angular Dev Server, que detecta que empieza con, /api y la redirige de http://localhost:4200/api/Fichajes a https://localhost:7286/api/Fichajes.

Fase 3: Recepción en Backend (ASP.NET Core)

Paso 8

Middleware de autenticación: ASP.NET Core ejecuta automáticamente el middleware de autenticación JWT antes de llegar al controller. Este middleware:

- Extrae el token del header Authorization
- Verifica la firma digital (HMAC-SHA256) con la clave secreta
- Comprueba que no haya expirado (claim exp)
- Valida emisor y audiencia
- Extrae los claims (empleadoId, rol, email) y los pone disponibles en User.Claims

Si alguna validación falla, el middleware responde automáticamente con 401 Unauthorized y el controller nunca se ejecuta.

Paso 9

Controller recibe la petición: El método FichajesController.Post() se ejecuta. Tiene el atributo [Authorize], lo que significa que solo se ejecuta si el JWT fue validado exitosamente en el paso anterior.

Paso 10

Extracción de empleadoId del token: El controller obtiene el ID del empleado del token JWT (NO del body de la petición):

Este es el empleadoId confiable porque viene del token firmado por el servidor, no del frontend donde podría ser manipulado.

Paso 11

Validaciones de negocio: El controller ejecuta validaciones que el frontend no puede hacer:

- Busca el último fichaje del empleado en la base de datos
- Verifica que el último fue una "Salida" (no permitir Entrada-Entrada)
- Valida que la fecha/hora sea lógica (no en el futuro)

Si alguna validación falla, responde con 400 Bad Request y un mensaje de error.

Fase 4: Persistencia en Base de Datos (PostgreSQL)

Paso 12

Creación de la entidad: El controller crea una instancia de la entidad Fichaje:

Paso 13

Entity Framework genera SQL: Entity Framework Core traduce la operación a una query SQL parametrizada:

Los parámetros @p0, @p1, etc. previenen ataques de SQL Injection porque el valor nunca se concatena directamente en la query.

Paso 14

PostgreSQL ejecuta INSERT: La base de datos:

- Ejecuta la inserción en la tabla Fichajes
- Genera automáticamente un ID (columna SERIAL PRIMARY KEY)
- Verifica las foreign keys (que el EmpleadoId exista en la tabla Empleados)
- Verifica los constraints (NOT NULL, tipos de datos)
- Persiste el registro en disco
- Devuelve el ID generado (ej: 123)

Fase 5: Respuesta al Frontend

Paso 15

Controller construye DTO de respuesta: El controller crea un objeto DTO con los datos del fichaje recién creado, incluyendo el ID generado por la base de datos:

Paso 16

HTTP 201 Created: El controller responde con código HTTP 201 (Created) y el DTO en formato JSON:

Paso 17

Observable notifica al componente:

Fase 6: Actualización de la Interfaz

Paso 18

Sincronización con backend: El componente actualiza el registro local con el ID del backend:

Esto permite futuras operaciones sobre ese fichaje (añadir observación, eliminar) usando el ID real de la base de datos.

Paso 19

Actualización de estado: El componente actualiza las propiedades que controlan la UI:

- lastType = 'Entrada' (muestra "Último fichaje: Entrada")
- lastTime = '09:30' (hora del fichaje)
- lastDate = '05/12/2025' (fecha del fichaje)
- entradaDisabled = true (botón Entrada deshabilitado)
- salidaDisabled = false (botón Salida habilitado)

Paso 20

Feedback visual: Se muestra un modal de éxito con el mensaje "Entrada registrada correctamente" y la hora del fichaje. El usuario cierra el modal haciendo click en "Aceptar".

Paso 21

Persistencia local: El historial actualizado se guarda nuevamente en localStorage para que persista si el usuario recarga la página:

Seguridad en Múltiples Capas

La seguridad del sistema no depende de una sola capa, sino que está distribuida en todas:

Capa 1: Frontend (Experiencia de Usuario)

- **Validaciones de formulario:** Reactive Forms con validadores de Angular verifican formato de email, longitud de contraseña, campos requeridos, etc. Esto proporciona feedback inmediato al usuario.
- **Guards de rutas:** authGuard impide navegar a rutas protegidas sin estar logueado. adminGuard impide acceso a rutas de administración sin el rol Admin.

- **Lógica de UI:** Botones deshabilitados según el estado (no permitir dos entradas seguidas visualmente).
- **Limitación:** Todo esto podría ser saltado con el DevTools del navegador, por lo que NO es seguridad real, solo UX.

Capa 2: Transporte HTTP

- **HTTPS en producción:** Encriptación TLS 1.3 previene que terceros lean las peticiones en tránsito (Man-in-the-Middle).
- **JWT con expiración:** Los tokens tienen un tiempo de vida limitado (8 horas). Después expiran y el usuario debe volver a loguearse.
- **Interceptor automático:** Añade el JWT sin intervención manual, reduciendo errores humanos.
- **Proxy en desarrollo:** Evita exponer configuraciones CORS innecesarias.

Capa 3: Backend (Validación Real)

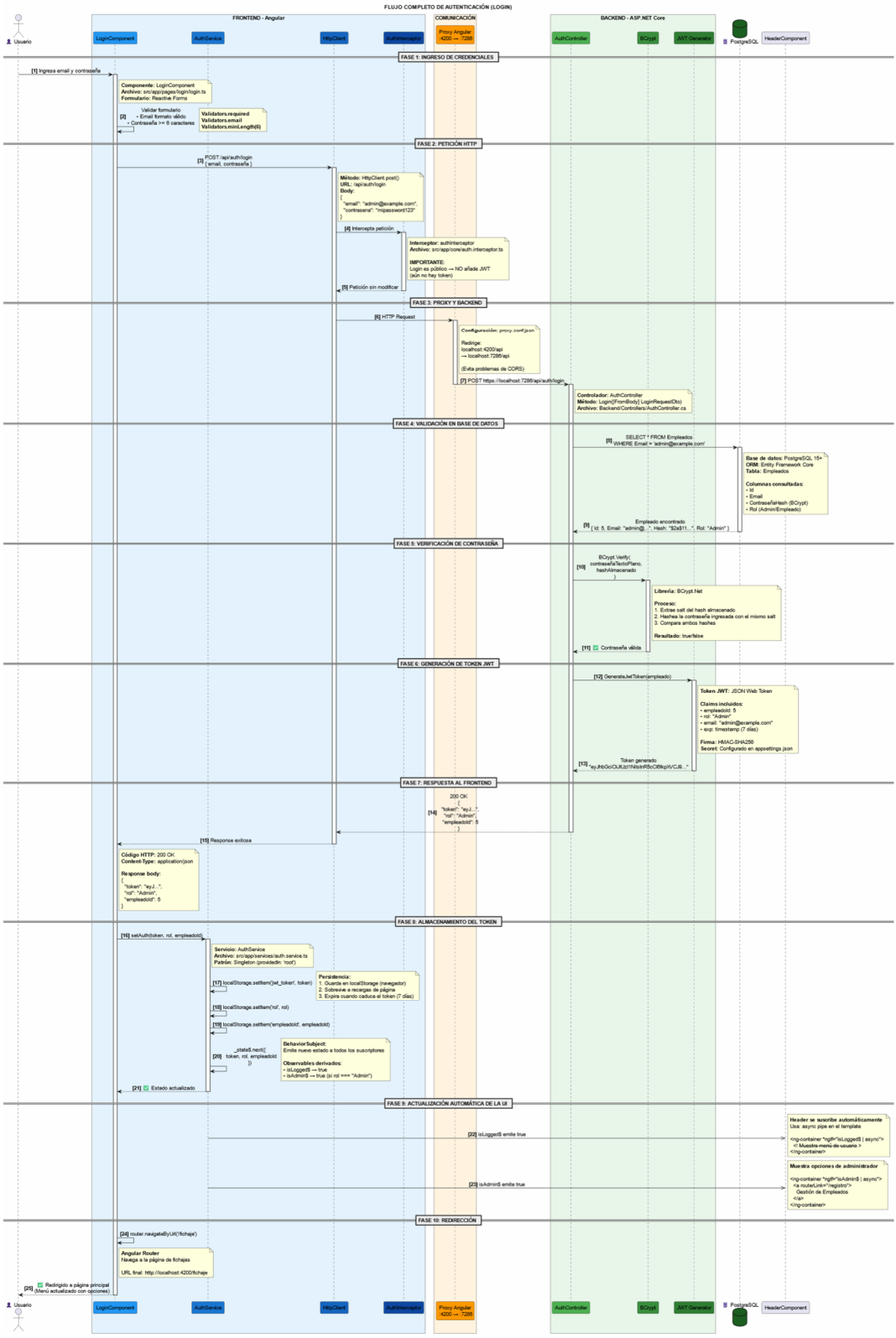
- **Atributo [Authorize]:** Todos los endpoints protegidos requieren JWT válido. Sin token válido, la petición ni siquiera llega al código del controller.
- **EmpleadoId del token:** El ID del empleado se extrae del token JWT firmado por el servidor, no del body de la petición que puede ser manipulado.
- **Validaciones de negocio:** El backend verifica reglas que el frontend no puede garantizar (última entrada tiene salida, no duplicados, permisos de rol).
- **Queries parametrizadas:** Entity Framework Core usa parámetros en las queries SQL, previniendo SQL Injection.
- **BCrypt para contraseñas:** Las contraseñas se almacenan hasheadas con BCrypt (irreversible), no en texto plano.

Capa 4: Base de Datos

- **Foreign Keys:** La columna EmpleadoId en Fichajes tiene una foreign key a Empleados.Id, garantizando que no se pueda crear un fichaje para un empleado inexistente.
- **Unique Constraints:** Email y DNI en Empleados son únicos, previniendo duplicados.

- **NOT NULL:** Campos críticos como Tipo y FechaHora en Fichajes no pueden ser nulos.
- **Tipos de datos:** PostgreSQL valida que los datos tengan el tipo correcto (no se puede insertar texto en una columna numérica).

DIAGRAMA DE FLUJO DE AUTENTICACIÓN



4. Backend

El backend ha sido desarrollado con .NET 8.

Los archivos mas importantes del backend son:

- program.cs
  - Para qué sirve: punto de entrada y bootstrap de la aplicación.
  - Contiene: configuración de DI, registro de **DbContext** (Npgsql), autenticación JWT, autorización, CORS, JSON options, Swagger, migraciones automáticas y seed del admin.
  - Clases/métodos clave: configuración de **TokenValidationParameters**, **db.Database.Migrate()**, registro de middlewares y **app.MapControllers()**.
- appsettings.json
  - Para qué sirve: configuración central (connection strings, JWT, logging, CORS).
  - Contiene: ConnectionStrings:ConnectionSql (Postgres), Jwt (Key, Issuer, Audience, ExpiresHours), Cors:Origins, niveles de Logging.
- Data\ApplicationDbContext.cs
  - Para qué sirve: capa de persistencia EF Core.
  - Contiene: DbSet<Empleado> Empleados, DbSet<Fichaje> Fichajes, DbSet<Ausencia> Ausencias.
- Models\Empleado.cs
  - Para que sirve: entidad de dominio Empleado.

```
using System.ComponentModel.DataAnnotations;

namespace FichajeApp.Models
{
    public class Empleado
    {
        [Key] public int Id { get; set; }

        [Required, MaxLength(100)] public string Nombre { get;
set; } = default!;
        [Required, MaxLength(150)] public string Apellidos {
get; set; } = default!;
        [Required, MaxLength(15)] public string DNI { get; set;
} = default!;
        [Required, EmailAddress, MaxLength(150)] public string
Email { get; set; } = default!;
```

```
        [Required, MaxLength(256)] public string Contrasena {
get; set; } = default!;
        [MaxLength(20)] public string? Telefono { get; set; }

        [Required] public Rol Rol { get; set; } = Rol.User;
    }
}
```

- Models\Ausencia.cs
    - Para qué sirve: entidad de solicitudes de ausencia.

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace FichajeApp.Models
{
    public class Ausencia
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }

        // Empleado que solicita la ausencia
        public int EmpleadoId { get; set; }
        public Empleado Empleado { get; set; } = default!;

        // Fecha de inicio y fin (almacenar en UTC)
        public DateTime FechaInicio { get; set; }
        public DateTime FechaFin { get; set; }

        [MaxLength(500)]
        public string Motivo { get; set; } = default!;

        // null = pendiente, true = aceptada, false = rechazada
        public bool? Aceptada { get; set; }
    }
}
```

- Models\Fichaje.cs
    - Para qué sirve: entidad que representa un registro de entrada/salida.

```
using System;
```

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace FichajeApp.Models
{
    public class Fichaje
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }

        // Relación con Empleado
        public int EmpleadoId { get; set; }
        public Empleado Empleado { get; set; } = default!;

        public DateTime FechaHora { get; set; } =
DateTime.UtcNow;

        [MaxLength(20)]
        public string Tipo { get; set; } = "Entrada"; //
"Entrada" | "Salida"

        [MaxLength(300)]
        public string? Observacion { get; set; }
    }
}
```

- Models\Rol.cs
    - Para qué sirve: enum de roles (Admin y User).

```
namespace FichajeApp.Models
{
    public enum Rol
    {
        Admin = 1,
        User = 2
    }
}
```

- Models\Dtos (varios archivos).
    - Para qué sirven: modelos para entrada/salida entre frontend y backend; evitan exponer entidades.

- Los archivos que contienen son: AusenciaDto.cs, AuthDto.cs, CreateEmpleadoDto.cs, CreateFichajeDto.cs, EmpleadoDto.cs, FichajeDto.cs, UpdateEmpleadoDto.cs y UpdateFichajeDto.cs
- Ejemplo de un archivo Dto comparado con el modelo
  El modelo representa la estructura de la base de datos.

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace FichajeApp.Models
{
    public class Ausencia
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }

        // Empleado que solicita la ausencia
        public int EmpleadoId { get; set; }
        public Empleado Empleado { get; set; } = default!;

        // Fecha de inicio y fin (almacenar en UTC)
        public DateTime FechaInicio { get; set; }
        public DateTime FechaFin { get; set; }

        [MaxLength(500)]
        public string Motivo { get; set; } = default!;

        // null = pendiente, true = aceptada, false = rechazada
        public bool? Aceptada { get; set; }
    }
}
```

En cambio el archivo AusenciaDto.cs sólo contiene los datos que el frontend necesita, sin datos sensibles como contraseñas.

```
using System;
using System.ComponentModel.DataAnnotations;

namespace FichajeApp.DTOs
{
    public class AusenciaDto
    {
        public int Id { get; set; }
```

```
        public int EmpleadoId { get; set; }
        public string? EmpleadoNombre { get; set; }
        public DateTime FechaInicio { get; set; }
        public DateTime FechaFin { get; set; }
        public string Motivo { get; set; } = default!;
        public bool? Aceptada { get; set; }
    }

    public class AusenciaCreateDto
    {
        [Required]
        public string FechaInicio { get; set; } = default!; //
aceptar texto desde el front

        [Required]
        public string FechaFin { get; set; } = default!; //
aceptar texto desde el front

        [Required]
        [MaxLength(500)]
        public string Motivo { get; set; } = default!;
    }

    public class AusenciaUpdateDto
    {
        public string? FechaInicio { get; set; } // opcional,
texto
        public string? FechaFin { get; set; }    // opcional,
texto

        [MaxLength(500)]
        public string? Motivo { get; set; }
    }

    public class AprobarRequest
    {
        public bool Aceptada { get; set; }
    }
}
```

- Controllers (varios archivos):
  En la carpeta Controllers están los controladores de API REST que definen todos los endpoints (rutas HTTP) del backend. Cada controlador gestiona un recurso específico del sistema.

- Controllers\AuthController.cs
  Endpoints de autenticación: login (emite JWT), posiblemente refresh token y endpoints públicos relacionados con auth.

  El endpoint que contiene es:
  POST /api/auth/login

  Ejemplo de uso:

  Angular hace la petición:
  this.http.post('/api/auth/login', { email: 'admin@app.com', contrasena: '123456' })

  AuthController responde, 200 OK { token: "eyJhbGc...", rol: "Admin", empleadoId: 1 } o 401 Unauthorized (credenciales incorrectas)

- Controllers\EmpleadosController.cs
  CRUD de empleados. Contiene lógica de creación, actualización, eliminación (soft-delete), y endpoints admin-only. Debe usar _db, CurrentUserId, IsAdmin y servicios como IEmpleadoExportService.

  Los endpoints que contiene son:
  GET /api/empleados → (Admin) Listar todos los empleados
  GET /api/empleados/{id} → (Admin) Un empleado por ID
  POST /api/empleados → (Admin) Crear empleado
  PUT /api/empleados/{id} → (Admin) Actualizar empleado
  DELETE /api/empleados/{id} → (Admin) Eliminar empleado

  Ejemplo de uso:

  Admin crea empleado con la petición:
  this.http.post('/api/empleados', {
  nombre: 'Juan',
  apellidos: 'García',
  email: 'juan@empresa.com',
  contrasena: 'Password123!',
  dni: '12345678A',

rol: 'User'
})

El backend, valida que sea Admin, hashea contraseña con BCrypt, guarda en BD y responde 201 Created (sin devolver contraseña)

- Controllers\FichajesController.cs
  Endpoints para fichajes: crear fichaje, listar (míos, por usuario, todos si admin), obtener por id, actualizar observación. Usa claims del JWT para seguridad y aplica filtros/joins con Empleado.

  Los endpoints que contiene son:

  GET /api/fichajes → Lista fichajes del usuario actual
  GET /api/fichajes/usuario/{id} → (Admin) Fichajes de empleado específico
  GET /api/fichajes/mios → Fichajes del token JWT
  GET /api/fichajes/{id} → Un fichaje por ID
  POST /api/fichajes → Crear fichaje (Entrada/Salida)
  PUT /api/fichajes/{id}/observacion → Añadir observación

  Ejemplo de uso:

  Angular hace la petición:
  this.http.post('/api/fichajes', { tipo: 'Entrada', fechaHora: '2024-12-06T10:30:00Z' })

  FichajesController, lee EmpleadoId del token JWT, crea fichaje con los datos enviados, guarda en la base de datos y responde 201 Created con los datos guardados.

- Controllers\AusenciasController.cs
  Endpoints para gestionar ausencias: solicitar, listar propias, listar por empleado (admin), aprobar/denegar y borrar.

  Los endpoints que contiene son:

GET /api/Ausencias → Ausencias del usuario
GET /api/Ausencias/usuario/{id} (Admin)Ausencias de un usuario
GET /api/Ausencias/{id} → (Admin) Obtener ausencia
POST /api/ausencias → Solicitar ausencia
PUT /api/ausencias/{id} → Actualizar ausencia
PUT /api/ausencias/{id}/aprobar → (Admin) Aprobar/rechazar
DELETE /api/ausencias/{id} → Eliminar ausencia

Ejemplo de uso:

Usuario solicita ausencia con la petición:

```
this.http.post('/api/ausencias', {
  fechaInicio: '2024-12-20',
  fechaFin: '2024-12-27',
  motivo: 'Vacaciones de Navidad'
})
```

Admin aprueba con la petición

```
this.http.patch('/api/ausencias/5/aprobar', { aceptada: true })
```

- Controllers\FichajesController.cs
  Endpoints para fichajes: crear fichaje, listar (míos, por usuario, todos si admin), obtener por id, actualizar observación. Usa claims del JWT para seguridad y aplica filtros/joins con Empleado.

  Los endpoints que contiene son:

  GET /api/fichajes → Lista fichajes del usuario actual
  GET /api/fichajes/usuario/{id} → (Admin) Fichajes de empleado específico
  GET /api/fichajes/mios → Fichajes del token JWT
  GET /api/fichajes/{id} → Un fichaje por ID
  POST /api/fichajes → Crear fichaje (Entrada/Salida)
  PUT /api/fichajes/{id}/observacion → Añadir observación

  Ejemplo de uso:

Angular hace la petición:
this.http.post('/api/fichajes', { tipo: 'Entrada', fechaHora: '2024-12-06T10:30:00Z' })

FichajesController, lee EmpleadoId del token JWT, crea fichaje con los datos enviados, guarda en la base de datos y responde 201 Created con los datos guardados

- Migrations (varios archivos)
  La carpeta con migraciones EF Core; representa cambios en el esquema de la base de datos, como pueden ser crear tabla, agregar o quitar columna.
- .gitignore
  Lista de archivos/carpetas que Git ignora (bin/, obj/, secrets, etc.).

5. Base de datos

La base de datos utiliza el sistema de gestión de datos relacional, PostgreSQL que almacena toda la información persistente de la aplicación: empleados, fichajes y ausencias.

Características principales:

- Base de datos relacional.
- Migraciones automáticas con Entity Framework Core.
- Relaciones con Foreign Keys y cascada.
- Índices para optimización de consultas.
- Constraints CHECK para validación a nivel BD.
- Timestamps con zona horaria UTC.

Relaciones entre tablas:

Empleados 1 – N Fichajes – Un empleado puede tener muchos fichajes y un fichaje, sólo puede tener un empleado.

Empleados 1 – N Ausencias – Un empleado puede tener muchas ausencias, pero una ausencia, sólo puede tener un empleado.

La tabla Empleados está compuesta por:

- “Id”, que es la clave primaria, es un int auto incrementable.
- “Nombre”, es de tipo varchar y longitud 100.
- “Apellidos”, es de tipo varchar y longitud 150.
- “Dni”, es de tipo varchar y longitud 15.
- “Email”, es de tipo varchar y longitud 150.
- “Contraseña”, es de tipo varchar y longitud 256.
- “Telefono”, es de tipo varchar y longitud 20.
- “Rol”, es de tipo varchar y longitud 50.

La tabla Ausencias está compuesta por:

- “Id”, que es la clave primaria, es un int auto incrementable.
- “EmpleadoId”, es de tipo int y es clave foránea.
- “FechaInicio”, es de tipo timestamp.
- “FechaFin”, es de tipo timestamp.
- “Motivo”, es de tipo varchar y longitud 500.
- “Aceptada”, es de tipo boolean.

La tabla Fichajes está compuesta por:

- “Id”, que es la clave primaria, es un int auto incrementable.
- “EmpleadoId”, es de tipo int y es clave foránea.
- “FechaHora”, es de tipo timestamptz.
- “Tipo”, es de tipo varchar y longitud 20.
- “Observacion”, es de tipo varchar y longitud 300.

DIAGRAMA ENTIDAD/RELACIÓN

